

Assignment 5: Research Assignment

Domain: Video game sprite (2D visual object) detection. Utilization of CNNs to enhance video game optimization, performance, and real-time object detection.

Focus: Object Detection.

Utilization of object detection in video games has multiple purposes. Real-time object detection that can enhance user experience by notifying a player if an object of interest is within view. Object detection can be used to enhance artificial intelligence in games by allowing the agent to interact with its environment more uniquely. CNNs can be used as a method to collect game play analytics by capturing objects that players interact with. There are many other use-cases for object detection in games.

It should be noted that this does come at a cost. There is computational overhead with utilizing AI models to perform complex tasks. While some of the functionalities, such as analytics, can be run on the backend, real-time application can lead to heavy resource usage and delays. Keeping the model lightweight and fast is important for optimization.

Data Set:

The data set utilized in this research was generated by us. Part of the research was learning how to create, label, and structure data for a CNN trained on object detection.

Samples were created using 18 background images and 4 foreground objects. Two of the foreground objects were designated as “targets” and 2 were designated as “distractors.”

Each background image and foreground image were fed into a processing pipeline to generate multiple variations of the images, diversifying the potential sets to provide a wider breadth of information for our model.

Image variation:

- 1) Raw images were resized. Background images were sized to 256x256 px. Foreground images were resized to 32x32 px.
- 2) Four variants were created for each bg image and each fg image. We took 40% darker, 20% darker, 20% brighter, and 40% brighter to generate the new image. This provided us with 5 images each: 1 original + 4 variations.
- 3) We took all of the resized images and variants and rotated 90, 180, and 270 degrees. This provided us with 4 rotational variants: 1 original + 3 variations.
- 4) We flipped all of the current images on the x-axis, y-axis, then both axes. This provided us with 4 inverted variants: 1 original + 3 variations.
- 5) We applied gaussian noise to some images to further vary the images. This step did not produce additional images; it modified randomly selected images from the current pools.

At the end, we had 1,440 background images and 640 foreground images to be used to generate samples.

Producing Samples:

- 1) The number of samples are passed as a parameter for the method. We chose 2,500 samples.
- 2) A background image is randomly selected from the pool.
- 3) Foreground images are self-divided. The first half of all foreground images are “targets,” and the second half are all “distractors.” Two “targets” are randomly selected, and 2 “distractors” are randomly selected. Each sample has exactly 4 objects with an even split of “targets” and “distractors.”
- 4) The 4 objects are randomly placed on the background image. A buffer is in place to ensure no overlap. The random positions include corners and edges.
 - a. Each object’s class and coordinates are captured to be placed in a label.
 - b. We have chosen a YOLO-label style which takes class, x-center, y-center, width, and height.
- 5) Generating samples includes multiple steps.
 - a. Generate the image as described. The sample is pooled into an image list.
 - b. Generate the label as described above. The label is pooled into a label list at the same position as the image.

- c. Once all the samples and labels are generated, they are written on disk. At this time, we provide names. We save images to an image subfolder and labels to a label subfolder.
 - i. We follow a naming convention of “sample_XXXXX.png” where each “x” is a digit starting from “00000” incrementing with each sample. Labels share identical names with extension “.txt”.
- 6) Generating metadata and datasplits.
- a. We create a “classes.txt” that is saved to a subfolder called “metadata”.
 - b. We generate datasplits that are saved to a subfolder called “datasplits”.
 - i. We employed a 70/15/15 split for training, validation, and testing.
 - ii. These are .txt files that have mappings for the images by name. The lists are randomized.



Figure 1: Example sample, sample_00066.png. We have two targets (elf and pig) and two distractors (dragon and monster) imposed at random positions on a randomly selected background.

```
0 0.621094 0.175781 0.125000 0.125000
0 0.542969 0.601562 0.125000 0.125000
1 0.785156 0.066406 0.125000 0.125000
1 0.468750 0.074219 0.125000 0.125000
```

Figure 2 Example label, sample_00066.txt. We have 4 entries in format (class, x-center, y-center, width, and height). Note there are 4 entries corresponding to 4 objects. Class 0 are targets and class 1 are distractors. The centers and width/height are normalized between 0 and 1.

```
target  
distractor
```

Figure 3: classes.txt representing "target" as 0 and "distractor" as 1.

```
dataset_2500/images/sample_00433.png  
dataset_2500/images/sample_02356.png  
dataset_2500/images/sample_01031.png  
dataset_2500/images/sample_02259.png  
dataset_2500/images/sample_01504.png  
dataset_2500/images/sample_00968.png  
dataset_2500/images/sample_01830.png  
dataset_2500/images/sample_00048.png  
dataset_2500/images/sample_02079.png  
dataset_2500/images/sample_02432.png  
dataset_2500/images/sample_01498.png
```

Figure 4: Example format for datasplit using train.txt. It includes the relative datapath along with the file name for proper mapping. The samples are randomized.

```
dataset_2500  
├── datasplits  
├── images  
├── labels  
├── metadata  
└── model
```

Figure 5: This is the structure for our dataset.

Implementation:

Model

We have elected to make a custom YOLO-based model. YOLO stands for “You Only Look Once” and is good for fast, lightweight implementations for object detection. Aside from the input layer, we have the following layers:

- 1) We have 4 convolutional layers with activation ‘relu’ and that progressively get broader by a factor of 2. There is MaxPooling after each. There is a bottleneck effect in this design that progressively reduces the effective size through MaxPooling, which assists with reducing overfitting while speeding up training. For object detection, this is significant as we home in on our target. This convolution and MaxPooling combination accounts for 8 layers.
- 2) We have a 5th convolutional layer that we use as a detection head. We set the kernel to 1 for pixel level detection.
- 3) We have a reshape layer. This is used to ensure proper YOLO shape.
- 4) We have 4 lambda functions. We need to detect our x-center and y-center, our width and height, our “objectness”, and our class. Our cxcy, obj, and cls utilize a sigmoid function to analyze our object capitalizing on the binary 0 and 1. We have applied an exponential function to ensure positive values for our width and height and to account for the normalization and variability of these values.
For a quick summary, we check the center position, expand the box to the correct size, ensure there is an object in the box, and then verify the class.
- 5) Finally, we have a Concatenate layer, which joins our lambda functions into the shape that our YOLO-type model needs.

This provides a total of 15 layers for our model: $8 + 1 + 1 + 4 + 1 = 15$

Code:

```
from tensorflow.keras import layers, models # type: ignore

def build_model(input_shape=(IMAGE_SIZE, IMAGE_SIZE, 3), S=16,
A=1):

    inputs = layers.Input(shape=input_shape)

# Backbone
```

```

    x = layers.Conv2D(32, 3, padding='same',
activation='relu')(inputs)

    x = layers.MaxPooling2D()(x)

    x = layers.Conv2D(64, 3, padding='same',
activation='relu')(x)

    x = layers.MaxPooling2D()(x)

    x = layers.Conv2D(128, 3, padding='same',
activation='relu')(x)

    x = layers.MaxPooling2D()(x)

    x = layers.Conv2D(256, 3, padding='same',
activation='relu')(x)

    x = layers.MaxPooling2D()(x) # Output shape: (S, S,
channels)

# Detection head: (S, S, A * 6)
out_channels = A * 6
x = layers.Conv2D(out_channels, 1, padding='same')(x)

# Reshape to (S, S, A, 6)
x = layers.Reshape((S, S, A, 6))(x)

# Now wrap each activation inside a Lambda layer
cxcy = layers.Lambda(lambda t: tf.sigmoid(t[...,
0:2]))(x) # center

wh = layers.Lambda(lambda t: tf.exp(t[..., 2:4]))(x)

obj = layers.Lambda(lambda t: tf.sigmoid(t[...,
4:5]))(x) # objectness

cls = layers.Lambda(lambda t: tf.sigmoid(t[...,
5:6]))(x) # class prob

```

```

output = layers.Concatenate(axis=-1)([cxcy, wh, obj, cls])

model = models.Model(inputs=inputs, outputs=output)

return model

model = build_model()
model.summary()

```

Focal Loss

This is a variation of a cross-entropy loss function. This is implemented to help reduce the weight of easy examples in an effort to allow our primary loss function to concentrate on more challenging examples. This plays an important role in a YOLO-style CNN because YOLO sacrifices thoroughness for speed, only conducting a single pass, which may lead to imbalance during training.

Code:

```

def focal_loss(y_true, y_pred, alpha=0.25, gamma=2.0):
    bce = tf.keras.backend.binary_crossentropy(y_true, y_pred)
    p_t = y_true * y_pred + (1 - y_true) * (1 - y_pred)
    alpha_factor = y_true * alpha + (1 - y_true) * (1 - alpha)
    modulating_factor = tf.pow(1.0 - p_t, gamma)
    return alpha_factor * modulating_factor * bce

```

Grid Loss

This is our primary loss function and follows a grid-like structure that is often needed for YOLO-style models. We are able to apply weights either by rewarding correct predictions or punishing incorrect predictions. It mirrors the shape what we have for our YOLO data, looking at x-center and y-center, width and height, objectness, and classification. These are then normalized, and we produce a “total loss” by dividing $[(\text{loc loss} + \text{obj loss} + \text{cls loss}) / \text{norm}]$.

Code:

```
def grid_loss(y_true, y_pred):

    # Predicted components
    pred_cxcy = y_pred[..., 0:2]
    pred_wh   = y_pred[..., 2:4]
    pred_obj  = y_pred[..., 4]
    pred_cls  = y_pred[..., 5]

    # Ground truth components
    true_cls  = y_true[..., 0]    # 1.0 = target, 0.0 =
distractor
    true_cxcy = y_true[..., 1:3]
    true_wh   = y_true[..., 3:5]
    true_obj  = y_true[..., 5]    # 1.0 = real object

    # --- Masks ---
    obj_mask = tf.cast(true_obj > 0.5, tf.float32)
    noobj_mask = 1.0 - obj_mask

    # --- Localization loss (only for objects) ---
```



```

    loc_loss = tf.reduce_sum(obj_mask[..., None] *
tf.square(true_cxcy - pred_cxcy))

    loc_loss += tf.reduce_sum(obj_mask[..., None] *
tf.square(true_wh - pred_wh))

    # --- Objectness loss ---

    bce_obj = tf.keras.backend.binary_crossentropy(true_obj,
pred_obj)

    obj_loss = tf.reduce_sum(obj_mask * bce_obj) +
tf.reduce_sum(noobj_mask * bce_obj)

    # --- Classification loss (for all objects) ---

    cls_focal = focal_loss(true_cls, pred_cls, alpha=0.25,
gamma=1.0)

    cls_loss = tf.reduce_sum(obj_mask * cls_focal)

    # --- Normalize by number of objects ---

    norm = tf.reduce_sum(obj_mask) + 1e-6

    total_loss = (loc_loss + obj_loss + cls_loss) / norm

    return total_loss

```

Compile Model

The model is compiled with Adam optimization and applying our grid loss.

Code:

```
model.compile(  
    optimizer=tf.keras.optimizers.Adam(),  
    loss=grid_loss  
)
```

Early Stopping

Early stopping is utilized to reduce overfitting. We will be basing our early stopping on our `val_loss` value, which is a loss score from our validation set. This is calculated during training along side the training set. By setting patience to 5, we allow 5 consecutive non-improving epochs before stopping the training. If we early stop, we will restore the best value from training at the time.

Code:

```
from tensorflow.keras.callbacks import EarlyStopping # type:  
ignore  
  
# Define early stopping  
early_stop = EarlyStopping(  
    monitor='val_loss',  
    patience=5,  
    restore_best_weights=True  
)
```

Training Model

This block of code trains our model with “fit”. We are utilizing our train_dataset for the primary training. Our val_dataset is used to calculate a metric called val_loss. The model conducts 20 iterations or “epochs” making predictions on images in the datasets and adjusting weights with the grid loss function. Early Stopping is used. Verbose allows a printout for each epoch.

Code:

```
history = model.fit(  
    train_dataset,  
    validation_data=val_dataset,  
    epochs=20,  
    callbacks=[early_stop],  
    verbose=1  
)
```

Testing

The model has been trained. There were some fine-tuning steps to find the optimal settings for score_threshold, nms_iou, and iou_threshold. We use our model with our test set and hyperparameter settings.

Code:

```
test_metrics = evaluate_predictions_with_nms(  
    model,  
    test_dataset,  
    score_threshold=0.0644,  
    nms_iou=0.70,  
    iou_threshold=0.5  
)
```

Results:

Metrics

True Positive = These are the predictions that correctly identify “targets.”

False Positives = These are the predictions that incorrectly identify background or “distractors” as “targets”.

False Negatives = These are the predictions that are incorrectly identified “targets” as not “targets”.

**True Negatives = True negatives are NOT used. This does not provide much value our instance of object detection. This encompasses everything in the background that is not an object, which is often uninformative to the model.

Precision = $TP / (TP + FP)$; This is a ratio of the model’s ability to correctly predict a “target” with its incorrect predictions of “targets.”

Recall = $TP / (TP + FN)$; This is a ratio of the model’s ability to correctly predict a “target” with its incorrect labeling of a “target” as a “distractor”.

Mean IoU = Mean Intersection of Union, which takes the average IoU across a dataset. This translates to how closely the model can draw a predicted bounding box against the ground truth bounding box, measuring the difference in the overlap.

Interpretation

```
Test Set Metrics:  
TP: 750  
FP: 0  
FN: 0  
Precision: 1.0000  
Recall: 1.0000  
Mean IoU: 0.9780307412147522
```

Figure 6: TP = True Positive, FP = False Positive, FN = False Negative, Precision, Recall, and Mean IoU (Intersection of Union). The model appears to have performed extremely well.



Figure 7: Randomly sample the test set to confirm our metrics. Red = Target, Blue = Distractor, Green = Prediction. We can visually see high precision, recall, and nearly perfect bounding boxes around the Targets.

Our model performed extremely well on our test dataset. Performing this well raises flags. We printed samples of the test set's predictions to verify that our statistics matched and were likely accurate.

The model predicted all 750 samples correctly, with no false negatives and no false positives. The results in prediction of 1.00 and recall of 1.00. However, our mean IoU is

marginally less than perfect with roughly 0.978. In general terms, the model can identify targets vs distractors with bounding boxes being largely correct.

The success of the model can likely be attributed to multiple factors.

- 1) The dataset is simplistic and there is little variability in the “targets” and “distractors.” All objects are of equal size, with clear boundaries, and are relatively unique images. Similarly, the simplistic nature of the dataset may have generated a test set that closely resembles the training and validation set, even though the specific samples were not included in the training nor validation set.
- 2) The combination of focal loss and grid loss quickly adjusts weights but still provides room to learn.
- 3) Fine-tuning was implemented post-training to discover optimal hyperparameters.
- 4) The evaluation focus is on correctly identifying an object while allowing some leeway for bounding boxes. There is a threshold for predicted bounding boxes for the object to count as a true positive.
- 5) The selected metrics provide adequate representation to evaluate the model’s performance, but the analysis could be more comprehensive.

Discussion

While the model performs extremely well with our given metrics, there are many limitations to this model against this dataset. This model is highly specialized at identifying two very specific targets. It will not likely do well identifying something unique that has been labeled a target. This is by designing the dataset and working through the process of data creation and implementing a model around it. In exploring these concepts, some aspects were oversimplified to achieve proof of concept.

The dataset and model choice were designed for fast, lightweight CNN. The samples are relatively small images at 256x256 px. This provides less surface area for the model to gather context but allows for less overhead for data storage. Due to the reduced sample size, the objects were also scaled down to 32x32 px. Again, this provides less context for the model to extrapolate information. Therefore, the model had to learn quickly and with a tolerable margin of error.

Utilizing a YOLO-style dataset and model provided a means for a fast and lightweight, yet efficient solution. A major consideration and what provides this system to work is the labels. YOLO labels contain the metadata to effectively get an objects classification, position, and box size.

Again, this model is very good at locating the very specific examples of targets. If this model were to be utilized in a real-world scenario, we would need to expand the dataset to include a wider variety of positive cases and negative cases for better generalization. Another likely use case is to incorporate the specialized model in a larger network. Many of these fast, lightweight models can be trained on specific targets that, when combined, make a more complex network capable of a greater capacity for object detection, or some other functionality that leverages this object detection.

This is a promising start, but this can be expanded upon. Using our trained model in a real-world situation would provide valuable insight into the validity of our methodology. Reevaluating our dataset's size and complexity to fit a direct example as opposed to our generic experimental would likely provide a stronger guiding principle to both dataset design and model design. Understanding the end goal for the model (standalone, component in a larger network, etc) should also play a factor in determining both dataset and model design.